Expectations, Outcomes, and Challenges of Modern Code Review

Nikhil Anand

March 23, 2026

This paper investigates how modern, tool-based code review is actually practiced in industry, focusing on developers at Microsoft. It examines the gap between what developers and managers expect from code reviews—primarily defect detection—and what reviews actually achieve in practice.

Using a mixed-method empirical approach that includes observations, interviews, comment analysis, and large-scale surveys, the authors find that modern code reviews provide value beyond bug detection. While finding defects remains the main motivation, most review activity centers on improving code quality, sharing knowledge, increasing team awareness, and generating alternative solutions. A key insight is that understanding code changes and their context is the central challenge in reviews, and current tools often fail to adequately support this need.

1 Motivation

The motivation for studying modern code review arises from its long-standing role as a key quality assurance practice in software engineering. Traditional code inspections, formalized by Fagan, were designed to systematically detect defects through structured, synchronous meetings, and have been shown to be effective in improving software quality [1, 2]. However, these approaches are often time-consuming and cumbersome, limiting their adoption in fast-paced industrial environments [3]. As a result, organizations have shifted toward more lightweight and tool-supported review practices.

This shift toward modern code review introduces uncertainty about whether prior knowledge from formal inspections still applies. Contemporary practices are informal, asynchronous, and heavily supported by tools, which fundamentally changes how developers interact with code and each other during reviews [4]. While these newer approaches improve efficiency and scalability, it is unclear whether they achieve the same goals as traditional inspections, particularly in terms of defect detection and overall code quality.

Additionally, modern code review is increasingly embedded in large-scale, collaborative development environments, where it serves not only as a defect detection mechanism but also as a medium for communication and coordination. Prior research suggests that review processes can capture valuable design rationale and support knowledge sharing among developers [5], and that developers often struggle with understanding code changes and team knowledge distribution [6]. These aspects highlight the broader socio-technical role of code review beyond simple error detection.

Given these evolving practices and expectations, there is a need for an empirical investigation into what developers and managers actually expect from code review, what outcomes are realized in practice, and what challenges arise. Understanding these factors can help practitioners better integrate code review into their workflows and guide researchers in improving tools and methodologies to better support modern development environments.

1.1 Related Work

Prior work on code review has largely focused on traditional code inspections and early forms of distributed review. For instance, studies on distributed and asynchronous inspections explored how tools could support geographically separated reviewers in identifying and discussing defects [7]. Surveys of inspection techniques have also provided taxonomies and comparative analyses of different approaches, highlighting factors such as team size, inspection type, and session structure in determining effectiveness

[8, 9]. Additionally, research examining open source development has shown how peer review influences project management decisions and collaborative workflows [10]. These studies, however, primarily examined formal or semi-formal inspection processes rather than modern, lightweight practices.

A key limitation of traditional inspection methods identified in prior research is their inefficiency, particularly due to the need for synchronous meetings and scheduling overhead. For example, it has been shown that a significant portion of inspection time can be wasted due to coordination delays [11]. This inefficiency motivated the development of early tool-based solutions such as ICICLE, CAIS, and Scrutiny, which aimed to enable asynchronous and more efficient review processes [12, 13, 14]. These systems laid the groundwork for modern code review tools by emphasizing collaboration, reduced overhead, and support for distributed teams.

More recent research has shifted toward understanding lightweight and tool-supported review practices, particularly in open source environments. Empirical studies of projects such as Apache have shown that modern reviews tend to be small, frequent, and involve brief, independent contributions from multiple reviewers [15]. Furthermore, research has highlighted that code review discussions often capture valuable design rationale, making them useful for future maintenance and comprehension tasks [5]. Complementary studies on developer behavior indicate that many challenges faced by developers stem from difficulties in understanding code changes and accessing team knowledge, reinforcing the importance of review processes in supporting comprehension [6].

Overall, while prior literature provides substantial insights into both traditional inspections and emerging lightweight practices, it does not fully address how modern, tool-based code review operates in large-scale industrial settings. In particular, there remains a gap in understanding how motivations, outcomes, and challenges of contemporary code review compare to earlier expectations, motivating further empirical investigation.

2 Methodology

2.1 Research Questions

The study is guided by three primary research questions that aim to explore the motivations, outcomes, and challenges of modern code review in practice:

1. What are the motivations and expectations for modern code review? Do they change from managers to developers and testers?
2. What are the actual outcomes of modern code review? Do they match the expectations?
3. What are the main challenges experienced when performing modern code reviews relative to the expectations and outcomes?

2.2 Research Setting

The study is conducted within a large industrial context at Microsoft, where software development spans diverse domains such as enterprise systems, mobile applications, and search engines. This diversity allows the authors to observe code review practices across teams with varying cultures, workflows, and policies, making the setting suitable for understanding modern code review in realistic, heterogeneous environments. The focus is on teams that have widely adopted a common tool for code review, CodeFlow, which is used by tens of thousands of developers. This tool represents a typical modern code review system, similar in functionality to other widely used platforms such as Mondrian, Phabricator, and Gerrit, enabling asynchronous, tool-based, and collaborative review processes.

CodeFlow structures the review process by allowing authors to submit a package of code changes along with a textual description, select reviewers, and initiate a discussion around the changes. Reviewers can inspect diffs, annotate code inline, and engage in threaded discussions that may occur synchronously or asynchronously. The system centralizes all review-related data, including comments and interactions, which provides a rich and unobtrusive source of empirical data for analysis without interfering with

developer behavior. This environment allows the researchers to study code reviews "in situ," capturing both the social and technical aspects of the practice as it naturally occurs in industry.

2.3 Research Method

The study adopts a mixed-methods approach, combining qualitative and quantitative techniques to triangulate findings and improve validity [16]. The methodology consists of multiple phases: (1) analysis of a prior internal study, (2) observations and interviews with developers, (3) card sorting of interview data, (4) card sorting of code review comments, (5) construction of an affinity diagram, and (6) large-scale surveys of managers and developers.

The first phase involves analyzing transcripts from an earlier study conducted at Microsoft, focusing on responses to questions about the goals of code review. Using coding techniques inspired by grounded theory, responses are broken into smaller units and grouped into concepts and categories [17, 18]. This initial analysis helps identify preliminary motivations for code review, which are later refined through subsequent phases.

The second phase consists of direct observations and semi-structured interviews with developers actively performing code reviews. Participants are selected across multiple teams and experience levels to ensure diversity. During each session, researchers first observe developers conducting real reviews, encouraging them to think aloud, and then follow up with interviews guided by evolving research questions. Semi-structured interviews are chosen to allow flexibility and exploration of emerging themes [19, 20]. The collected data is transcribed and segmented into meaningful units for further analysis.

To organize and abstract the qualitative data, the researchers employ open card sorting, a technique commonly used in information architecture to derive themes and hierarchies from unstructured data [21]. This process is applied both to interview data (over a thousand coded units) and to a sampled set of 570 code review comments, enabling the identification of recurring patterns and categories. The resulting categories are then synthesized using an affinity diagram, which groups related concepts and helps derive higher-level themes through collaborative analysis [22].

Finally, the study validates and generalizes its findings through two surveys: one targeting managers and another targeting developers. The surveys are designed following established guidelines for opinion-based empirical studies [23]. The manager survey gathers qualitative insights into motivations for code review, while the developer survey uses structured questions to quantify the importance of different motivations and experiences. High response rates from both groups strengthen the reliability of the results. By combining these diverse data sources, the methodology ensures a comprehensive and well-supported understanding of modern code review practices.

3 Key Findings

The paper aims to answer three research questions regarding the motivations, outcomes, and challenges of modern code review. The key findings are organized around these questions, revealing insights into the complex role that code review plays in contemporary software development.

3.1 Why do Programmers do Code Review?

Finding Defects Finding defects emerges as the primary and most widely agreed-upon motivation for code review among both developers and managers. Many practitioners view code review as an efficient way to catch bugs early, sometimes even considering it cheaper than testing for certain issues. This includes both low-level logical errors and higher-level design flaws. However, while this motivation is dominant in perception, it does not fully capture the breadth of what code reviews accomplish in practice. Developers often approach reviews expecting to identify flaws in logic or correctness, but this expectation represents only one dimension of the activity.

Code Improvement Code improvement is a closely ranked motivation, focusing on enhancing readability, maintainability, and adherence to coding standards rather than correctness. Reviewers frequently

comment on formatting, naming conventions, and removal of redundant or unused code. This type of feedback is often the first pass during review and is considered essential for maintaining consistency across a team. However, the paper highlights a subtle concern: because these improvements are easier to identify, reviewers may focus disproportionately on them, sometimes overlooking deeper issues such as architectural flaws or security concerns.

Alternative Solutions This motivation involves suggesting better or more efficient ways to implement a solution. Developers value the opportunity to receive feedback that leads to improved designs or alternative approaches, reflecting the collective intelligence of the team. However, there is a notable disconnect between developers and managers on this aspect—developers consider it moderately important, while managers rarely emphasize it. In practice, suggestions for alternative solutions tend to be less frequent and often remain at a high level rather than deeply explored.

Knowledge Transfer Code review serves as a significant mechanism for learning and knowledge sharing within teams. Both reviewers and authors benefit from exposure to different parts of the codebase, APIs, and design decisions. This bidirectional learning helps distribute expertise, onboard new developers, and reinforce best practices. Interestingly, this motivation often emerges organically rather than being explicitly planned, indicating that developers perceive code review as an educational process embedded within their workflow.

Team Awareness and Transparency Another important motivation is maintaining awareness of ongoing changes within the team. Code reviews act as a communication channel that keeps team members informed about new features, modifications, and design decisions. This transparency helps prevent unexpected changes, fosters alignment, and enables developers to stay updated on relevant parts of the system. It also allows team members to discover new functionalities that they can reuse or integrate into their own work.

Share Code Ownership Closely related to awareness, shared code ownership emphasizes collaboration and reduces reliance on individual experts. Through code review, multiple developers become familiar with different parts of the codebase, mitigating knowledge silos and increasing system resilience. This also has a cultural impact, making developers less protective of their code and more open to feedback. Over time, this fosters a sense of collective responsibility and encourages a more collaborative development environment.

Summary Overall, while defect detection is the most prominent stated motivation, code review fulfills a much broader role in practice. It supports code quality, learning, communication, and collaboration within teams. The motivations span both technical and social dimensions, highlighting that modern code review is not just a verification activity but a central component of team coordination and knowledge sharing.

3.2 The Outcomes of Code Review

The paper presents a detailed empirical analysis of what actually happens during modern code review, contrasting sharply with earlier expectations. By examining 570 review comments across multiple threads, the authors categorize and quantify the types of feedback given during reviews. The findings reveal that the most common outcome of code review is not defect detection, but rather code improvement. Nearly one-third of all comments focus on improving code practices, readability, and removing unnecessary or redundant code. These types of changes are relatively easy to identify and act upon, which likely contributes to their high frequency. This establishes code review as a mechanism primarily used for refining and polishing code rather than deeply analyzing correctness.

In contrast, defect detection—despite being the most frequently cited motivation—accounts for a significantly smaller portion of review outcomes. Only a small fraction of comments are related to defects, and even within this category, most issues are minor and localized. These include logical mistakes, simple configuration errors, or overlooked corner cases. High-level issues such as architectural flaws, security

vulnerabilities, or complex design problems are rarely identified during reviews. This indicates that while developers expect code review to uncover meaningful defects, in practice it tends to surface only superficial problems.

The authors explore the reasons behind this mismatch between expectations and outcomes. One key explanation lies in the nature of modern, tool-based reviews. Reviewers often prioritize quick, visible issues—such as formatting or small logic errors—over deeper analysis, partly because these are easier to detect and require less contextual understanding. Interviews suggest that reviewers sometimes default to commenting on minor issues when they lack sufficient familiarity with the code. Additionally, some reviewers acknowledge that focusing on easy improvements can mask the presence of more significant problems, which may go unnoticed.

Another important observation is that the ability to detect meaningful defects is closely tied to the reviewer's level of understanding of the code and its context. When reviewers lack familiarity with the codebase or the purpose of the changes, their feedback tends to remain shallow. This reinforces the idea that deeper insights require not only time but also contextual knowledge, which is often limited in modern, distributed review environments. As a result, the depth and quality of review outcomes vary significantly depending on the reviewer's expertise and involvement with the code.

Although less prominent, the study also finds evidence of knowledge transfer occurring during reviews. Some comments direct authors to documentation or suggest learning resources, indicating that reviews occasionally serve as opportunities for sharing expertise. However, such outcomes are relatively rare in the recorded data, likely because many learning interactions happen outside the review tool, such as through direct communication.

Overall, the paper concludes that modern code review does not fully meet its primary expectation of defect detection. Instead, it functions more effectively as a tool for incremental code improvement and team-level knowledge exchange. The discrepancy between expected and actual outcomes is not due to flawed practices but rather reflects the inherent limitations of lightweight, tool-based reviews. These findings highlight the need to reconsider the role of code review within the broader development process and suggest that it should be complemented with other techniques to ensure comprehensive defect detection.

3.3 Challenges of Code Review

Code Review is Understanding The central challenge identified in this section is that code review is fundamentally an exercise in understanding. Across interviews and observations, developers consistently report that the most difficult aspect of reviewing code is grasping the rationale behind changes—why the change was made, what problem it solves, and how it fits into the broader system. Even when descriptions are provided, they are often insufficient or incomplete, leaving reviewers to infer intent from the code itself. This lack of clarity makes it difficult to provide meaningful feedback. As a result, much of the review effort is spent not on evaluating correctness, but on reconstructing context. This explains why understanding-related comments (questions, clarifications) are highly prevalent and why deeper issues are often missed—without sufficient understanding, reviewers cannot effectively assess design or correctness.

The authors also highlight that understanding is the dominant factor influencing the time and quality of reviews. Other challenges such as scheduling or time pressure are often secondary and can be traced back to the effort required to comprehend the code. Developers indicate that understanding consumes the majority of review time, reinforcing that improving comprehension is key to improving review effectiveness. This insight reframes code review from a verification task into a cognitive task centered on information processing and context-building.

A Priori Understanding The authors emphasize the importance of prior familiarity with the codebase in determining review effectiveness. Developers who are already familiar with the files under review—such as code owners or contributors to that component—are able to review faster and provide more insightful feedback. Their comments tend to be deeper, more conceptual, and more actionable, often identifying subtle issues or suggesting better design alternatives. In contrast, reviewers unfamiliar

with the code must invest additional time to understand its purpose, dependencies, and constraints, which often limits their feedback to surface-level observations.

The findings show that unfamiliarity significantly increases the effort required for review. Most developers report that reviewing unfamiliar code requires exploring additional context beyond the changes themselves, such as reading related files or understanding system behavior. Even then, their understanding remains shallow compared to that of experienced contributors. This results in feedback that is less confident, less detailed, and focused on easily observable aspects like style or syntax. The subsection clearly establishes that prior knowledge is a critical enabler of effective code review.

Dealing with Understanding Needs Given the challenges of understanding, developers adopt various strategies to gather the necessary context. These include reading change descriptions, exploring related code, executing the code, consulting documentation, and directly communicating with the author or other team members. In many cases, reviewers resort to synchronous communication—such as face-to-face discussions or real-time messaging—to clarify doubts more efficiently. This highlights a limitation of current code review tools, which primarily support asynchronous, text-based interactions and offer limited assistance for deeper comprehension.

The authors underscore that existing tools provide only basic features such as diff views, inline comments, and syntax highlighting, which are insufficient for supporting complex understanding tasks. As a result, reviewers must rely on external resources and communication channels to fill the gaps. This fragmented approach to understanding reduces efficiency and may lead to incomplete or superficial reviews. The findings suggest that improving tool support for context and comprehension could significantly enhance the effectiveness of code review, enabling reviewers to move beyond surface-level feedback and address more complex issues.

4 Implications

The paper concludes by translating its empirical findings into a set of actionable recommendations for practitioners and broader implications for researchers. These are grounded in the observed mismatch between expectations and outcomes of code review, and the central role of understanding in shaping review effectiveness.

4.1 Recommendations for Practitioners

4.1.1 Rethinking Code Review as a Quality Assurance Mechanism

One of the primary recommendations is that teams should reconsider relying on code review as a strong defect detection mechanism. The study shows that reviews tend to uncover only superficial issues rather than deep or critical defects. As a result, treating code review as a primary line of defense for quality assurance can be misleading. Instead, teams should complement code reviews with other practices such as automated testing, static analysis, and formal verification techniques to ensure comprehensive defect detection.

4.1.2 Improving Reviewer Understanding

Since understanding is identified as the key challenge in code review, improving the reviewer's context is critical. The paper recommends that teams ensure reviewers have sufficient familiarity with the code being reviewed. This can be achieved by involving code owners or developers with relevant expertise in the review process. Additionally, authors should provide clear, detailed descriptions of changes, including the rationale and expected impact, to reduce the cognitive burden on reviewers. Enhancing understanding directly improves the depth and usefulness of feedback.

4.1.3 Recognizing Benefits Beyond Defect Detection

The study emphasizes that code review provides multiple benefits beyond finding bugs, such as improving code quality, facilitating knowledge transfer, increasing team awareness, and encouraging shared

ownership. Teams should explicitly acknowledge and leverage these broader benefits when designing their review processes. For instance, reviews can be structured to encourage discussion, learning, and collaboration rather than focusing solely on correctness.

4.1.4 Enhancing Communication During Reviews

Another key recommendation is to improve communication mechanisms within the review process. While modern tools support asynchronous interactions, they often fall short in enabling rich, high-bandwidth communication. Developers frequently resort to in-person discussions or synchronous messaging to clarify complex issues. Teams should therefore incorporate opportunities for direct communication—either through integrated tools or organizational practices—to support better understanding and more effective reviews.

4.2 Implications for Researchers

4.2.1 Opportunities for Automating Routine Review Tasks

The findings suggest that a significant portion of review effort is spent on identifying code improvements and simple defects, which are relatively straightforward and repetitive. This presents an opportunity for research in automated tools that can handle such tasks, such as enforcing coding standards, detecting common bugs, or identifying dead code. By automating these low-level checks, developers can focus their attention on more complex and meaningful aspects of code review, such as design and architecture.

4.2.2 Advancing Program Comprehension Support

A major implication is the need for better tool support for code understanding. Despite extensive research in program comprehension and the availability of advanced features in modern IDEs, current code review tools provide only minimal support for understanding code changes. This gap highlights a significant opportunity for researchers to design tools that integrate contextual information, dependency analysis, and historical insights directly into the review process. Improving comprehension support could directly address the core challenge identified in the study.

4.2.3 Exploring Socio-Technical Aspects of Code Review

The study reveals that code review has important social and collaborative dimensions, such as knowledge sharing, team awareness, and cultural shifts toward shared ownership. However, these aspects are difficult to measure and remain underexplored. This opens up avenues for research into the socio-technical effects of code review, including how it influences team dynamics, learning, and long-term productivity. Developing methods to capture and evaluate these outcomes could provide a more holistic understanding of the value of code review.

4.2.4 Bridging the Gap Between Expectations and Practice

Finally, the mismatch between expected and actual outcomes of code review highlights a broader research challenge: aligning tools and practices with real-world needs. Researchers are encouraged to focus on practical problems faced by developers, particularly those related to understanding and collaboration. By addressing these gaps, future work can help evolve code review into a more effective and impactful practice in modern software development.

Overall, the recommendations and implications underscore a shift in perspective—from viewing code review as a defect-finding activity to recognizing it as a multifaceted process that combines technical evaluation with communication, learning, and collaboration.

5 Limitations

The paper acknowledges that, as a qualitative study, there are inherent challenges in establishing the validity and generalizability of its findings. Although the authors attempt to mitigate this through

triangulation—combining observations, interviews, surveys, and analysis of review comments—the interpretation of qualitative data can still introduce subjectivity. The study relies on coding, categorization, and thematic analysis, which depend on researcher judgment, and therefore may be influenced by bias despite efforts to ensure consistency and agreement among researchers [24].

Another limitation concerns the context of the study, which is conducted entirely within Microsoft. While the company provides a diverse set of teams, projects, and development practices, findings from a single organization may not generalize to other industrial or open-source environments. Differences in team dynamics, project size, or development culture could lead to different code review behaviors. However, the authors argue that insights from single-case studies can still contribute meaningfully to scientific understanding, as supported by prior research in case study methodology [25, 26].

The study also highlights limitations related to the observability of certain outcomes. Some motivations and benefits of code review—such as knowledge transfer, team awareness, and learning—are inherently difficult to measure through artifacts like review comments. While survey responses and interviews suggest that these outcomes exist, there is limited "hard evidence" to quantify their frequency or impact. This means that some conclusions rely on self-reported perceptions rather than directly observable data.

Finally, the analysis of outcomes is based largely on review comments recorded in the CodeFlow tool, which may not capture the full extent of interactions during code review. Developers often engage in offline or synchronous communication, such as face-to-face discussions or real-time messaging, especially when addressing complex understanding needs. These interactions are not fully represented in the dataset, potentially leading to an incomplete picture of the review process and its outcomes.

References

[1] Michael E. Fagan. "Design and code inspections to reduce errors in program development". In: *IBM Systems Journal* (1976).

[2] A. Frank Ackerman, Priscilla J. Fowler, and Robert G. Ebenau. "Software inspections and the industrial production of software". In: *Proceedings of a symposium on Software validation: inspection-testing-verification-alternatives*. 1984, pp. 13–40.

[3] Forrest Shull. "Inspecting the history of inspections". In: *IEEE Software* (2008).

[4] Peter Rigby et al. "Open source peer review—lessons and recommendations for closed source". In: *IEEE Software* (2012).

[5] Alistair Sutherland and Gina Venolia. "Can peer code reviews be exploited for later information needs?" In: *Proceedings of the International Conference on Software Engineering*. 2009.

[6] Thomas LaToza, Gina Venolia, and Robert DeLine. "Maintaining mental models: a study of developer work habits". In: *Proceedings of the International Conference on Software Engineering*. 2006.

[7] Mark Stein et al. "Distributed software inspection". In: 1997.

[8] Oliver Laitenberger. "A survey of software inspection technologies". In: *Handbook on Software Engineering and Knowledge Engineering*. 2002, pp. 517–555.

[9] Adam Porter, Harvey Siy, and Lawrence Votta. "A review of software inspections". In: *Advances in Computers* 42 (1996), pp. 39–76.

[10] Justin P. Johnson. "Collaboration, peer review, and open source software". In: *Information Economics and Policy* 18 (2006), pp. 477–497.

[11] Lawrence G. Votta. "Does every inspection need a meeting?" In: *ACM SIGSOFT Software Engineering Notes* 18 (1993), pp. 107–114.

[12] Lisa Brothers, V. Sembugamoorthy, and Michael Muller. "ICICLE: groupware for code inspection". In: *Conference on Computer-Supported Cooperative Work*. 1990, pp. 169–181.

[13] Vahid Mashayekhi, Chris Feulner, and John Riedl. "CAIS: collaborative asynchronous inspection of software". In: *SIGSOFT Software Engineering Notes* 19 (1994), pp. 21–34.

[14] John Gintell et al. "Scrutiny: A collaborative inspection and review system". In: *Proceedings of the 4th European Software Engineering Conference*. 1993, pp. 344–360.

[15] Peter Rigby, Daniel German, and Margaret-Anne Storey. "Open source software peer review practices: a case study of the Apache server". In: *Proceedings of ICSE*. 2008.

[16] John W. Creswell. *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*. Sage, 2009.
[17] Bruce L. Berg. *Qualitative Research Methods for Social Sciences*. Pearson, 2004.
[18] Steve Adolph, Wendy Hall, and Philippe Kruchten. "Using grounded theory to study the experience of software development". In: *Empirical Software Engineering* 16.4 (2011), pp. 487–513.
[19] Thomas Lindlof and Bryan Taylor. *Qualitative Communication Research Methods*. Sage, 2002.
[20] Robert S. Weiss. *Learning From Strangers: The Art and Method of Qualitative Interview Studies*. Free Press, 1995.
[21] Ian Barker. *What is information architecture?* 2005.
[22] Stuart J. Janis et al. *Improving Performance Through Statistical Thinking*. McGraw-Hill, 2000.
[23] Barbara A. Kitchenham and Shari L. Pfleeger. "Personal opinion surveys". In: *Guide to Advanced Empirical Software Engineering*. Springer, 2007.
[24] Nahid Golafshani. "Understanding reliability and validity in qualitative research". In: *The Qualitative Report* 8 (2003), pp. 597–607.
[25] Bent Flyvbjerg. "Five misunderstandings about case-study research". In: *Qualitative Inquiry* 12.2 (2006), pp. 219–245.
[26] Victor Basili, Forrest Shull, and Filippo Lanubile. "Building knowledge through families of experiments". In: *IEEE Transactions on Software Engineering* 25.4 (1999), pp. 456–473.